

LD11 Master Theory A-to-Z

Detailed theory document covering MVD and all missed uploaded-source topics.

Field	Value
Student	Ruturaj Mokashi
Topic	Learning Day 11 - Hyperparameter Tuning
Document type	Individual PDF documentation
Date	2026-06-14

Submission note
This is an individual PDF document. The documentation package intentionally does not include a combined/merged PDF.

1. What Hyperparameter Tuning Means

A model has two kinds of values: parameters and hyperparameters. Parameters are learned from data. Hyperparameters are chosen before the model learns, or chosen by a search procedure during tuning. In KNN, the number of neighbours is a hyperparameter. In Ridge Regression, alpha is a hyperparameter. In Random Forest, tree count, maximum depth, and minimum leaf size are hyperparameters. The purpose of tuning is to find settings that generalise well, not settings that only look good on the training data.

In a business context, hyperparameter tuning is like testing different operating policies before selecting one. The best policy is not the one that works only on historical examples. It is the one that also works on future data. That is why validation and test separation is non-negotiable.

2. Parameters vs Hyperparameters

Parameters are internal values learned by the algorithm. Logistic Regression learns coefficients. Ridge Regression learns coefficients. Decision Trees learn split thresholds. Neural networks learn weights. These are not manually selected row by row. They are estimated during training.

Hyperparameters control how the model learns. They shape model flexibility, regularisation, runtime, and risk of overfitting. A small K in KNN can overfit because predictions depend on very few neighbours. A very high alpha in Ridge can underfit because coefficients are shrunk too strongly. A very deep tree can memorize noise. Tuning manages these trade-offs.

3. Train, Validation, and Test Logic

The training set is where the model learns parameters. Validation is where we compare hyperparameter choices. The test set is saved for final evaluation. If the test set is used repeatedly during tuning, it stops being a fair test because modelling decisions become indirectly influenced by it.

Cross-validation improves validation reliability by using multiple folds. Instead of trusting one validation split, the algorithm trains and validates several times across different folds. The average score becomes the search score. This is useful when datasets are small or class balance matters.

4. Grid Search

Grid Search tries every combination from a manually defined grid. It is transparent and easy to explain. For example, if we try three values for KNN neighbours and two values for weights, Grid Search tests six combinations. This makes it good for teaching, small experiments, and audit-friendly workflows.

The weakness is scalability. If we add many hyperparameters and many possible values, the number of combinations grows quickly. Grid Search can waste time testing unimportant dimensions with the same intensity as important ones.

5. Random Search

Random Search samples combinations from a search space instead of testing all combinations. It is especially useful when some hyperparameters matter much more than others. Random Search may find good values faster because it explores more diverse combinations.

Random Search is not as systematic as Grid Search, but it is often more efficient in high-dimensional spaces. It is also a good baseline before using more advanced search methods like Bayesian optimisation, Hyperopt, or Optuna.

6. KNN Tuning

KNN is a distance-based algorithm. It predicts a new example by looking at nearby training examples. The main tuning question is how many neighbours should vote. A small number of neighbours makes the model sensitive to local noise. A large number smooths the decision boundary but can underfit.

Scaling is essential for KNN. If one feature has values from 0 to 10000 and another from 0 to 1, the larger-range feature dominates distance. That is why the notebook uses StandardScaler inside a Pipeline before tuning KNN.

7. Logistic Regression Tuning

Logistic Regression is simple, fast, and explainable. Its key tuning parameter is C, which controls regularisation strength. Smaller C means stronger regularisation. Larger C means weaker regularisation. Regularisation reduces overfitting by discouraging extreme coefficients.

The extended notebook uses GridSearchCV to compare C values and penalties. Logistic Regression became the best classification model in the extended notebook, showing that a simple model can outperform more complex methods when the data structure fits it well.

8. Ridge Regression Tuning

Ridge Regression is used for regression problems. It adds L2 regularisation to linear regression. The main hyperparameter is alpha. A low alpha behaves more like ordinary linear regression. A high alpha shrinks coefficients more strongly.

The extended notebook uses the diabetes dataset and tunes alpha using cross-validation. Evaluation uses RMSE, MAE, and R2. RMSE penalises large errors strongly, MAE gives average absolute error, and R2 shows the proportion of variance explained.

9. CASH

CASH means Combined Algorithm Selection and Hyperparameter Optimization. It asks two questions at once: which model family should we use, and which hyperparameters should that model use? This is broader than tuning one model.

In the notebook, CASH is implemented as a practical model family comparison. KNN, Logistic Regression, Random Forest, and SVC are compared using cross-validated ROC-AUC. Logistic Regression wins in this dataset. This is a simple, explainable CASH-style workflow.

10. Hyperopt

Hyperopt is an optimisation framework where we define an objective function and a search space. The objective function trains a model and returns a value to minimise, often negative validation score or error. The search algorithm then proposes new settings.

The important concept is that model tuning becomes an optimisation problem. Hyperopt can search over integers, floats, categories, and conditional spaces. It is useful when manual grid design becomes too limited.

11. Optuna

Optuna is another modern hyperparameter optimisation tool. It organises optimisation into studies and trials. In each trial, Optuna suggests parameter values, trains a model, evaluates it, and records the result.

Optuna is popular because it is flexible, readable, and supports pruning. Pruning stops weak trials early, which saves computation. In the extended notebook, Optuna is used safely for a small Random Forest search.

12. Bayesian Optimisation

Bayesian optimisation is designed for expensive experiments. Instead of randomly trying settings forever, it builds a surrogate model that estimates how the objective behaves. It also estimates uncertainty. This allows the optimiser to choose the next trial more intelligently.

The two competing ideas are exploration and exploitation. Exploration means trying uncertain areas. Exploitation means testing near known strong areas. A good optimiser balances both.

13. Acquisition Functions

The acquisition function decides where Bayesian optimisation should test next. Probability of Improvement chooses points likely to beat the current best. Expected Improvement considers both chance and size of improvement. Upper Confidence Bound combines predicted score and uncertainty.

In practical terms, the acquisition function is the decision rule that says: based on what we know so far, this next trial is worth running.

14. Neural Architecture Search

Neural Architecture Search means tuning the structure of a neural network. Architecture includes number of layers, number of neurons, activation function, dropout, and sometimes optimiser and learning rate.

NAS has three parts: search space, search strategy, and performance estimation. The search space defines possible architectures. The search strategy chooses what to test. Performance estimation decides how much training each architecture receives before judging it.

15. Multi-Fidelity Optimisation

Multi-fidelity optimisation saves time by evaluating some candidates cheaply first. A candidate might train for fewer epochs, use less data, or receive fewer resources. Weak candidates are stopped early. Strong candidates receive more resources.

Successive Halving and Hyperband are key examples. This is important because full model training can be expensive, especially for neural networks or large ensemble methods.

16. AutoML

AutoML automates parts of the machine learning workflow. It can include preprocessing selection, model family selection, hyperparameter tuning, validation, ensembling, and sometimes report generation. Auto-sklearn, TPOT, Hyperopt-Sklearn, and Auto-WEKA are examples from the broader AutoML ecosystem.

AutoML is not magic. It still needs correct data preparation, fair validation, sensible metrics, and business interpretation. AutoML can search faster than a human, but the analyst must still decide whether the output is suitable and trustworthy.

17. Theory Summary Table

Theory area	Key point	Practical notebook link
Classical tuning	Grid and Random Search are transparent search baselines.	KNN, Logistic Regression, Ridge.
Model family selection	CASH compares algorithm families and hyperparameters together.	CASH section.
Advanced optimisation	Hyperopt and Optuna turn tuning into objective optimisation.	Hyperopt/Optuna RF sections.
Bayesian ideas	Use surrogate model, uncertainty, and acquisition functions.	Theory and main notebook convergence sections.
Multi-fidelity	Spend more resources only on promising candidates.	Successive Halving section.
AutoML and NAS	Automate model/pipeline/architecture choices.	Theory coverage and documentation.

Master theory takeaway
A complete Day 11 understanding requires both practical tuning workflows and deeper source topics such as CASH, Hyperopt, Optuna, Bayesian multi-fidelity optimisation.